Laboratory Assignment File

for

Cryptography Laboratory (CS0557)

Master of Technology

in

Computer Science & Engineering

Submitted by

Yugal

(Roll no. 25903053)

Submitted to

Dr. Harsh Kumar Verma

(Assistant Professor, NIT Jalandhar)

Department of Computer Science & Engineering

Dr. B R Ambedkar National Institute of Technology Jalandhar

Punjab, India-144008

May, 2026

Contents

Assignment 1: Basic Programs (28-01-2026) 1

1.1 Write a program to print prime numbers between 1 and 1000. 1

1.2 Write a program to perform matrix multiplication. 1

1.3 Write a program to find inverse of a matrix. 4

Assignment 2: Caesar Substitution Cipher (04-02-2026) 7

2.1 Implement Caesar substitution cipher in Python. 7

2.2 Implement Hill cipher in Python. 8

List of Figures

1 Program to print prime numbers between 1 and 1000 1
2 Program to perform matrix multiplication 4
3 Program to find inverse of a matrix 6
4 Program to implement Caesar Substitution Cipher 8
5 Program to implement Hill Cipher 10

Assignment 1: Basic Programs

1.1 Write a program to print prime numbers between 1 and 1000.

```java
import java.util.ArrayList;
import java.util.List;

public class Prime {
    public static void main(String[] args) {
        List<Integer> primes = new ArrayList<>();
        int x = 1;
        int y = 100;

        for (int i = 2; i <= y; i++) {
            boolean isPrime = true;

            for (int j = 2; j <= i / 2; j++) {
                if (i % j == 0) {
                    isPrime = false;
                    break;
                }
            }

            if (isPrime) {
                primes.add(i);
            }
        }

        System.out.println("Prime numbers between " + x + " and " + y + ":");
        System.out.println(primes);
    }
}
```

Output

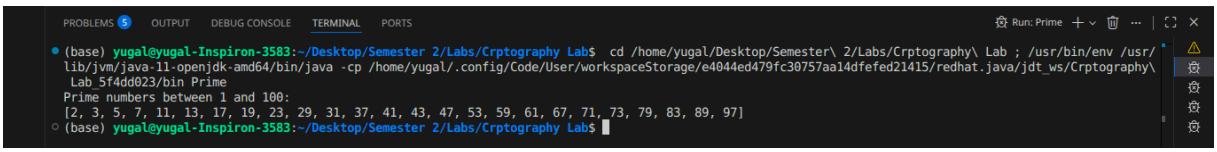


Figure 1: Program to print prime numbers between 1 and 1000

1.2 Write a program to perform matrix multiplication.

```java
import java.util.Scanner;

public class Matrix_Multiplication {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter the number of rows for the first matrix (A):");
        int rowsA = scanner.nextInt();
        System.out.println("Enter the number of columns for the first matrix (A):");
        int colsA = scanner.nextInt();

        System.out.println("Enter the number of rows for the second matrix (B):");
        int rowsB = scanner.nextInt();
        System.out.println("Enter the number of columns for the second matrix (B):");
        int colsB = scanner.nextInt();

        if (colsA != rowsB) {
            System.out.println("\nError: Matrix multiplication is not possible.");
            System.out.println("The number of columns in the first matrix must equal the number of rows in the second matrix.");
            return;
        }

        int[][] matrixA = new int[rowsA][colsA];
        int[][] matrixB = new int[rowsB][colsB];
        int[][] resultMatrix = new int[rowsA][colsB];

        System.out.println("\nEnter elements of the first matrix (A):");
        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsA; j++) {
                System.out.print("A[" + i + "][" + j + "]: ");
                matrixA[i][j] = scanner.nextInt();
            }
        }

```

```java
        System.out.println("\nEnter elements of the second matrix (B):");
        for (int i = 0; i < rowsB; i++) {
            for (int j = 0; j < colsB; j++) {
                System.out.print("B[" + i + "][" + j + "]: ");
                matrixB[i][j] = scanner.nextInt();
            }
        }

        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                for (int k = 0; k < colsA; k++) {
                    resultMatrix[i][j] += matrixA[i][k] * matrixB[k][j];
                }
            }
        }

        System.out.println("\nResultant Matrix:");
        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                System.out.print(resultMatrix[i][j] + " ");
            }
            System.out.println();
        }

        scanner.close();
    }
}
```

Output

```
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$  cd /home/yugal/Desktop/Semester\ 2/Labs/Crptography\
 Lab ; /usr/bin/env /usr/lib/jvm/java-11-openjdk-amd64/bin/java -cp /home/yugal/.config/Code/User/workspaceStorage/e4044ed479fc30
757aa14dfefed21415/redhat.java/jdt_ws/Crptography\ Lab_5f4dd023/bin Matrix_Multiplication
Enter the number of rows for the first matrix (A):
3
Enter the number of columns for the first matrix (A):
3
Enter the number of rows for the second matrix (B):
3
Enter the number of columns for the second matrix (B):
3

Enter elements of the first matrix (A):
A[0][0]: 1
A[0][1]: 2
A[0][2]: 3
A[1][0]: 4
A[1][1]: 5
A[1][2]: 6
A[2][0]: 7
A[2][1]: 8
A[2][2]: 9

Enter elements of the second matrix (B):
B[0][0]: 1
B[0][1]: 2
B[0][2]: 3
B[1][0]: 4
B[1][1]: 5
B[1][2]: 6
B[2][0]: 7
B[2][1]: 8
B[2][2]: 9

Resultant Matrix:
30 36 42
66 81 96
102 126 150
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$ ^C
```

Figure 2: Program to perform matrix multiplication

1.3 Write a program to find inverse of a matrix.

```java

import java.util.Scanner;

public class MatrixInverseSimple {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        double[][] a = new double[3][3];

        System.out.println("Enter elements of 3x3 matrix:");

        // Input
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                a[i][j] = sc.nextDouble();
            }
```

```java
        }

        // Determinant
        double det =
                a[0][0] * (a[1][1]*a[2][2] - a[1][2]*a[2][1])
              - a[0][1] * (a[1][0]*a[2][2] - a[1][2]*a[2][0])
              + a[0][2] * (a[1][0]*a[2][1] - a[1][1]*a[2][0]);

        if (det == 0) {
            System.out.println("Matrix is singular. No inverse.");
            return;
        }

        // Adjoint matrix
        double[][] adj = new double[3][3];

        adj[0][0] =  (a[1][1]*a[2][2] - a[1][2]*a[2][1]);
        adj[0][1] = -(a[0][1]*a[2][2] - a[0][2]*a[2][1]);
        adj[0][2] =  (a[0][1]*a[1][2] - a[0][2]*a[1][1]);

        adj[1][0] = -(a[1][0]*a[2][2] - a[1][2]*a[2][0]);
        adj[1][1] =  (a[0][0]*a[2][2] - a[0][2]*a[2][0]);
        adj[1][2] = -(a[0][0]*a[1][2] - a[0][2]*a[1][0]);

        adj[2][0] =  (a[1][0]*a[2][1] - a[1][1]*a[2][0]);
        adj[2][1] = -(a[0][0]*a[2][1] - a[0][1]*a[2][0]);
        adj[2][2] =  (a[0][0]*a[1][1] - a[0][1]*a[1][0]);

        // Inverse = adjoint / determinant
        System.out.println("Inverse Matrix:");

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.printf("%6.2f ", adj[i][j] / det);
            }
            System.out.println();
        }

        sc.close();
    }
}
```

Output

```
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$  cd /home/yugal/Desktop/Semester\ 2/Labs/Crptography\
 Lab ; /usr/bin/env /usr/lib/jvm/java-11-openjdk-amd64/bin/java -cp /home/yugal/.config/Code/User/workspaceStorage/e4044ed479fc30
757aa14dfefed21415/redhat.java/jdt_ws/Crptography\ Lab_5f4dd023/bin MatrixInverseSimple
Enter elements of 3x3 matrix:
2 3 5
6 7 3
4 5 9
Inverse Matrix:
 -2.40   0.10   1.30
  2.10   0.10  -1.20
 -0.10  -0.10   0.20
```

Figure 3: Program to find inverse of a matrix

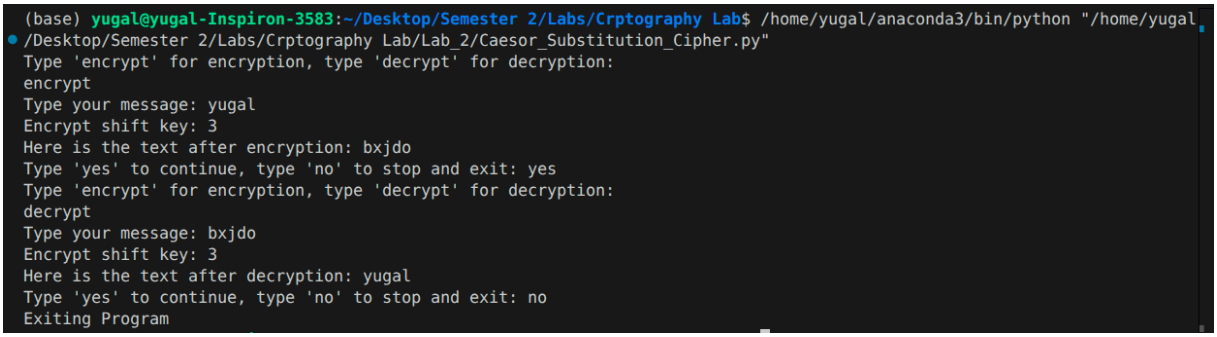
Assignment 2: Caesar Substitution Cipher

2.1 Implement Caesar substitution cipher in Python.

```python
alphabet=list('abcdefghijklmnopqrstuvwxyz')

def encryption(plain_text, shift_key):
    cipher_text = ""
    plain_text = plain_text.lower()
    for char in plain_text:
        if char in alphabet:
            position = alphabet.index(char)
            new_position = (position+shift_key)%26
            cipher_text += alphabet[new_position]
        else:
            cipher_text += char
    print(f"Here is the text after encryption: {cipher_text}")

def decryption(cipher_text, shift_key):
    plain_text = ""
    cipher_text = cipher_text.lower()
    for char in cipher_text:
        if char in alphabet:
            position = alphabet.index(char)
            new_position = (position-shift_key)%26
            plain_text += alphabet[new_position]
        else:
            plain_text += char
    print(f"Here is the text after decryption: {plain_text}")

wanna_end = False
while not wanna_end:
    what_to_do = input("Type 'encrypt' for encryption, type 'decrypt' for decryption: \n")
    text = input("Type your message: ")
    shift = int(input("Encrypt shift key: "))

    if what_to_do == "encrypt":
        encryption(plain_text=text, shift_key=shift)
    elif what_to_do == "decrypt":
        decryption(cipher_text=text, shift_key=shift)
    play_again = input("Type 'yes' to continue, type 'no' to stop and exit: ")

```

```
    if play_again == 'no':
        wanna_end = True
        print("Exiting Program")
```

Output



```
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$ /home/yugal/anaconda3/bin/python "/home/yugal
/Desktop/Semester 2/Labs/Crptography Lab/Lab_2/Caesor_Substitution_Cipher.py"
Type 'encrypt' for encryption, type 'decrypt' for decryption:
encrypt
Type your message: yugal
Encrypt shift key: 3
Here is the text after encryption: bxjdo
Type 'yes' to continue, type 'no' to stop and exit: yes
Type 'encrypt' for encryption, type 'decrypt' for decryption:
decrypt
Type your message: bxjdo
Encrypt shift key: 3
Here is the text after decryption: yugal
Type 'yes' to continue, type 'no' to stop and exit: no
Exiting Program
```

Figure 4: Program to implement Caesar Substitution Cipher

2.2 Implement Hill cipher in Python.

```
alphabet = list('abcdefghijklmnopqrstuvwxyz')

print("Enter 4 numbers for the key matrix (2*2): ")
a = int(input("a: "))
b = int(input("b: "))
c = int(input("c: "))
d = int(input("d: "))

key_matrix = [[a,b],
              [c,d]]

# modular inverse of number mod 26
def mod_inverse(num):
    num = num%26
    for i in range(26):
        if(num * i) % 26 == 1:
            return i
    return -1

# inverse key matrix
def inverse_key_matrix():
    a = key_matrix[0][0]
    b = key_matrix[0][1]
    c = key_matrix[1][0]
    d = key_matrix[1][1]
```

```python

    # determinant
    det = (a*d - b*c)%26

    inv_det = mod_inverse(det)
    if inv_det == -1:
        print("Invalid key!")
        return None

    # inverse matix
    inv_a = (d*inv_det)%26
    inv_b = (-b*inv_det)%26
    inv_c = (-c*inv_det)%26
    inv_d = (a*inv_det)%26

    return [[inv_a, inv_b],
            [inv_c, inv_d]]

def encrypt(plain_text):
    plain_text = plain_text.replace(" ","").lower()

    # if odd, use x
    if len(plain_text)%2 != 0:
        plain_text += 'x'

    cipher_text = ""

    # process letter in pair
    for i in range(0, len(plain_text), 2):
        p1 = alphabet.index(plain_text[i])
        p2 = alphabet.index(plain_text[i+1])

        # matrix muliplication
        c1 = (key_matrix[0][0] * p1 + key_matrix[0][1] * p2) % 26
        c2 = (key_matrix[1][0] * p1 + key_matrix[1][1] * p2) % 26

        cipher_text += alphabet[c1] + alphabet[c2]
    print("Encrypted text : ", cipher_text)

def decrypt(cipher_text):
    inv_matix = inverse_key_matrix()
    if inv_matix is None:
        return
    plain_text = ""
```

```python

    for i in range(0, len(cipher_text), 2):
        c1 = alphabet.index(cipher_text[i])
        c2 = alphabet.index(cipher_text[i+1])

        # multiply by inverse key
        p1 = (inv_matix[0][0] * c1 + inv_matix[0][1] * c2) % 26
        p2 = (inv_matix[1][0] * c1 + inv_matix[1][1] * c2) % 26

        plain_text += alphabet[p1] + alphabet[p2]
    print("Decrypted text: ", plain_text)

choice = input("Type 'encrypt' or 'decrypt': ")
message = input("Type your message: ")

if choice == "encrypt":
    encrypt(message)
elif choice == "decrypt":
    decrypt(message)
else:
    print("Invalid option")
```

Output

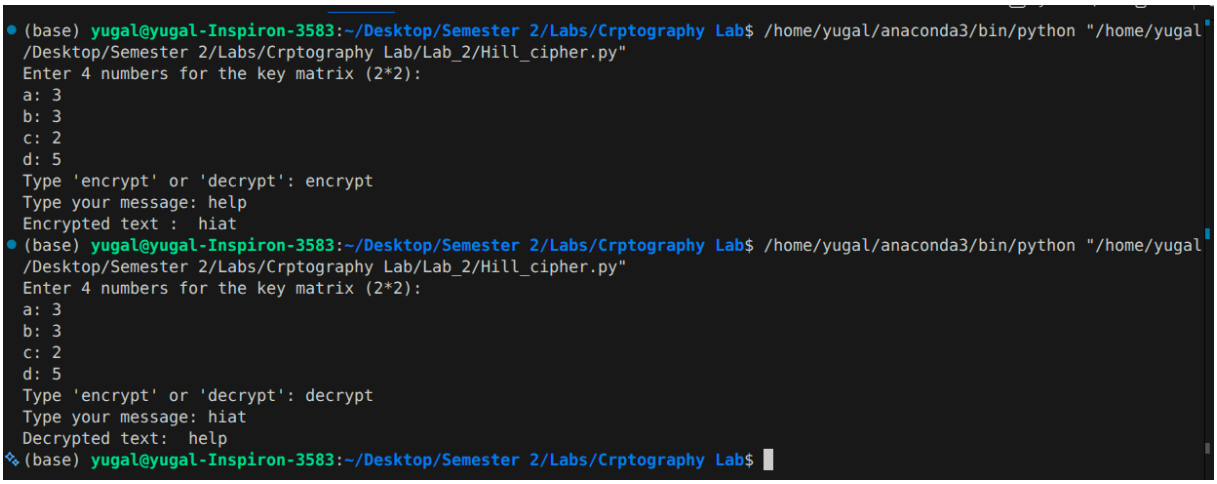


Figure 5: Program to implement Hill Cipher